

Calculus for Machine Learning

A Day 0E companion — from the slope of a line to the gradient of a matrix

KP Regression Book · Chapter 0 supplement

July 2026

Contents

1	How to read this	3
2	Part 1 — The derivative	4
2.1	1.1 A straight line has one slope, and that is the whole story	4
2.2	1.2 A curve does not have one slope	4
2.3	1.3 The secant trick	4
2.4	1.4 Why it was always going to be $6 + h$	5
2.5	1.5 The limit definition	5
2.6	1.6 Code proof: watch it converge	5
2.7	1.7 Break it deliberately: h too small	6
2.8	1.8 The practical rule	6
2.9	Exercises, Part 1	7
3	Part 2 — The rules	8
3.1	2.1 The power rule, derived rather than asserted	8
3.2	2.2 Constant multiple and sum rules	8
3.3	2.3 Code proof: rule versus measurement	9
3.4	2.4 The chain rule	9
3.5	2.5 The chain rule on the squared error	9
3.6	2.6 Break it deliberately: the dropped inner derivative	10
3.7	Exercises, Part 2	11
4	Part 3 — The functions machine learning actually differentiates	12
4.1	3.1 The exponential	12
4.2	3.2 The logarithm	12
4.3	3.3 The sigmoid, derived in full	12
4.4	3.4 Vanishing gradients, met early	13
4.5	3.5 tanh	13
4.6	3.6 ReLU, and a function that has no derivative at a point	13
4.7	3.7 Softplus, and a small delight	14
4.8	Exercises, Part 3	14
5	Part 4 — Curvature: the second derivative	15
5.1	4.1 The derivative of the derivative	15

5.2	4.2 Convexity: bowls and everything else	15
5.3	4.3 Why this is the whole ballgame	15
5.4	4.4 Taylor expansion: everything looks like a parabola if you zoom in	15
5.5	Exercises, Part 4	16
6	Part 5 — Many dials at once	17
6.1	5.1 Partial derivatives	17
6.2	5.2 The gradient	17
6.3	5.3 The Jacobian	18
6.4	5.4 The Hessian	18
6.5	5.5 Positive semidefinite, decoded	18
6.6	5.6 The multivariable chain rule, and the shape of backpropagation	19
6.7	Exercises, Part 5	19
7	Part 6 — Matrix calculus and the normal equations	20
7.1	6.1 Differentiating with respect to a vector	20
7.2	6.2 Four identities you will use forever	20
7.3	6.3 Deriving the normal equations, start to finish	20
7.4	6.4 Break it deliberately: collinearity	22
7.5	6.5 Ridge, as a one-line consequence	22
7.6	Exercises, Part 6	22
8	Part 7 — Optimisation, which is what calculus is <i>for</i>	23
8.1	7.1 Critical points	23
8.2	7.2 Gradient descent	23
8.3	7.3 Break it deliberately: the learning rate	23
8.4	7.4 How large is too large, exactly	24
8.5	7.5 Variants, named only	24
8.6	7.6 Constrained optimisation, briefly	25
8.7	Exercises, Part 7	25
9	Part 8 — Doing calculus without doing calculus	26
9.1	8.1 Forward versus central differences	26
9.2	8.2 The gradient check — build this and keep it	26
9.3	8.3 Symbolic differentiation with SymPy	27
9.4	8.4 A tiny autodiff engine, from scratch	27
9.5	8.5 Integration, the minimum viable dose	28
9.6	Exercises, Part 8	29
10	Appendix A — Formula sheet	30
11	Appendix B — Notation	30
12	Appendix C — Error decoder	31
13	Appendix D — Exit checks	31

1 How to read this

This document covers every piece of calculus this book uses, from a standing start. It assumes you can read Python and that you have met summation notation (Day 0C) and variance (Day 0D). It assumes nothing else.

The contract, same as the rest of the book:

1. Every symbol gets unpacked and connected to a number you can see.
2. Every claim is confirmed by code you can run.
3. You build something small in each part, and you break something on purpose.
4. Type the code. Do not copy it. Your fingers need this as much as your eyes do.

Every number printed in this document was produced by actually running the code shown. Where a result is exact, it is written exactly. Where floating point makes it messy, the messy value is shown rather than a tidied-up lie.

Suggested pace: Parts 1–4 in one sitting, Parts 5–6 in a second, Parts 7–8 in a third. Do the exercises between sittings, not after all of them.

Set up once:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install numpy sympy matplotlib
```

2 Part 1 — The derivative

2.1 1.1 A straight line has one slope, and that is the whole story

Start with something you already trust. Here is a straight-line model of project cost:

$$\text{cost} = 1.1 \cdot \text{cable_km} + 2$$

The slope is 1.1: each extra kilometre of cable adds 1.1 million PKR. Measure it between cable lengths of 10 km and 30 km — the cost rises from 13 to 35, so the slope is $22/20 = 1.1$. Measure it between 5 km and 25 km instead — cost rises from 7.5 to 29.5, so the slope is $22/20 = 1.1$ again.

That is what “straight” means. One number describes the whole line, and it does not matter where you stand when you measure it.

2.2 1.2 A curve does not have one slope

Now take $y = x^2$ and try the same thing.

- Between $x = 0$ and $x = 1$: rise = 1, run = 1, slope = 1.
- Between $x = 1$ and $x = 2$: rise = 3, run = 1, slope = 3.
- Between $x = 3$ and $x = 4$: rise = 7, run = 1, slope = 7.

Three answers. So the question “*what is the slope of $y = x^2$?*” has no answer — it is a badly formed question. The well-formed version is:

What is the slope of this curve **at one specific point**?

Answering that is the entire job of a derivative. Everything below is machinery for answering it.

2.3 1.3 The secant trick

Here is the difficulty stated plainly: a slope needs two points, and we want the slope at one.

So we cheat. Fix the point we care about — call it A , at $x = 3$, so $A = (3, 9)$. Put a second point B a small distance h further along the curve, at $x = 3 + h$. Two points, so we can draw a line through them. That line is called a **secant**, and it has a slope we can compute:

$$\text{secant slope} = \frac{f(3+h) - f(3)}{h}$$

With $h = 1$, that is $(16 - 9)/1 = 7$. Not the answer we want — the secant cuts through the curve rather than touching it at A . But now shrink h :

h	secant slope
1.00	7.00
0.50	6.50
0.25	6.25
0.10	6.10
0.01	6.01

As B slides toward A , the secant pivots, and its slope closes in on 6. When B finally arrives at A , the line stops cutting the curve and merely touches it. That limiting line is the **tangent**, and its slope — 6 — is the slope of the curve at $x = 3$.

2.4 1.4 Why it was always going to be $6 + h$

The animation-in-your-head is convincing, but four lines of ordinary algebra make it certain:

$$\begin{aligned}\text{secant slope} &= \frac{(3+h)^2 - 3^2}{h} \\ &= \frac{9 + 6h + h^2 - 9}{h} \\ &= \frac{6h + h^2}{h} \\ &= 6 + h\end{aligned}$$

Look at the last line. The secant slope is *exactly* $6 + h$, always. So of course it approached 6 — it was 6 plus a quantity we were deliberately shrinking to nothing. Check it against the table above: $h = 0.25$ gives 6.25; $h = 0.01$ gives 6.01. Every row matches.

There is no mystery here, and there never was one.

2.5 1.5 The limit definition

Everyone writes this the same way, so decode it once and be done:

$$\frac{dy}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Symbol	Said aloud	Means
$f(x+h) - f(x)$	“f of x plus h, minus f of x”	the rise
h	“h”	the run
$\lim_{h \rightarrow 0}$	“the limit as h goes to zero”	shrink the gap to nothing
$\frac{dy}{dx}$	“dee y by dee x”	the answer: the slope at a point

You have already watched every one of those four pieces happen. That is the complete definition of a derivative — there is no second, more advanced one waiting for you later.

Alternative notation you will meet: $f'(x)$ means the same thing as $\frac{dy}{dx}$. Use whichever is less cluttered in context.

2.6 1.6 Code proof: watch it converge

```
def f(x):
    return x ** 2
```

```
def numerical_slope(f, x, h):
    return (f(x + h) - f(x)) / h

x = 3.0
for h in [1.0, 0.1, 0.01, 0.0001, 0.000001]:
    print(f"h={h:<10} slope estimate={numerical_slope(f, x, h):.6f}")
```

Output:

```
h=1.0          slope estimate=7.000000
h=0.1          slope estimate=6.100000
h=0.01         slope estimate=6.010000
h=0.0001       slope estimate=6.000100
h=1e-06        slope estimate=6.000001
```

Notice `numerical_slope` never mentions x^2 . Hand it any function and it measures that function's slope. You have just written a tool, not an example.

2.7 1.7 Break it deliberately: h too small

The story so far says smaller h is better. Take that seriously and see what happens:

```
for h in [1e-8, 1e-10, 1e-12, 1e-14, 1e-16]:
    print(f"h={h:<8} slope estimate={numerical_slope(f, 3.0, h):.6f}")
```

Output:

```
h=1e-08    slope estimate=5.999999963535174
h=1e-10    slope estimate=6.000000496442226
h=1e-12    slope estimate=6.000533403494046
h=1e-14    slope estimate=6.217248937900877
h=1e-16    slope estimate=0.0
```

The estimates stop improving, start wobbling, and finally collapse to exactly zero — a slope of zero for a curve that is visibly climbing.

Here is why. A `float64` stores about 16 significant digits. Near 3.0, the smallest distinguishable step is roughly 4.4×10^{-16} . So when $h = 10^{-16}$, the expression `3.0 + h` rounds to exactly 3.0. The numerator becomes $3^2 - 3^2 = 0$, and zero divided by anything is zero.

The mathematics was never wrong. The arithmetic ran out of room. This distinction will matter repeatedly.

2.8 1.8 The practical rule

For numerical slopes, use $h \approx 10^{-6}$. Too large and the secant is not close enough to the tangent. Too small and floating-point noise swamps the signal.

This is not a deep truth; it is an engineering fact worth memorising. Part 8 turns it into a reusable tool, and Chapter 1 Day 5 uses that tool to check a hand-derived matrix gradient.

2.9 Exercises, Part 1

1.1 [core] Write `numerical_slope` from scratch without looking back. Use it on $f(x) = x^3$ at $x = 2$, where the true slope is 12. Sweep h from 10^{-1} down to 10^{-14} and report which h gets closest.

1.2 [core] Using the algebra of section 1.4 as a template, show by hand that the secant slope of $y = x^2$ at a general point x is $2x + h$.

1.3 [stretch] Repeat 1.2 for $y = x^3$. You should get $3x^2 + 3xh + h^2$. Confirm that as $h \rightarrow 0$ this leaves $3x^2$, and check it numerically at $x = 2$.

3 Part 2 — The rules

Computing a limit by hand every time would be miserable. Fortunately, doing it *once* in general produces shortcuts.

3.1 2.1 The power rule, derived rather than asserted

Run section 1.4's algebra with a general x instead of 3:

$$\frac{(x+h)^2 - x^2}{h} = \frac{x^2 + 2xh + h^2 - x^2}{h} = \frac{2xh + h^2}{h} = 2x + h \xrightarrow{h \rightarrow 0} 2x$$

Now do the identical thing for x^3 :

$$\frac{(x+h)^3 - x^3}{h} = \frac{3x^2h + 3xh^2 + h^3}{h} = 3x^2 + 3xh + h^2 \xrightarrow{h \rightarrow 0} 3x^2$$

Two from the square, three from the cube. A pattern:

$$\boxed{\frac{d}{dx} x^n = n x^{n-1}}$$

n	derivative
2	$2x^1$
3	$3x^2$
4	$4x^3$
1	$1x^0 = 1$
0	0

The last row deserves a moment. $y = x^0 = 1$ is a flat horizontal line. It never changes, so its slope is zero everywhere. A derivative measures change, and a constant does not change.

3.2 2.2 Constant multiple and sum rules

$$\frac{d}{dx} [a \cdot f(x)] = a \cdot \frac{d}{dx} f(x)$$

Multiply a function by 3 and its slope is multiplied by 3. Plot $y = x^2$ and $y = 3x^2$ together and look at the tangents at $x = 1$: one has slope 2, the other slope 6.

$$\frac{d}{dx} [f(x) + g(x)] = \frac{d}{dx} f(x) + \frac{d}{dx} g(x)$$

If a function is two things added together, differentiate them separately and add the results. You may go term by term.

Together:

$$y = 3x^2 + 5x \implies \frac{dy}{dx} = 6x + 5$$

3.3 2.3 Code proof: rule versus measurement

```
def y(x):
    return 3 * x**2 + 5 * x

def dy_dx_by_rule(x):
    return 6 * x + 5

print(dy_dx_by_rule(2.0))
print(numerical_slope(y, 2.0, h=0.000001))
```

Output:

```
17.0
17.000003005307462
```

Build this habit now and keep it for life. Whenever you derive a derivative by hand, check it numerically. It costs three lines and catches almost every mistake you will make. Note the numerical answer is not exactly 17 — that residue is the h in $6x + h$ -style leftovers plus floating-point noise, which is why we compare with a tolerance rather than `==`.

3.4 2.4 The chain rule

This is the most important rule in the document. Everything about neural networks, every activation function, and the whole of backpropagation is this one rule applied repeatedly.

Consider $E = (10 - (2m + 1))^2$. To evaluate it you do two things in order: first compute $10 - (2m + 1)$, then square the result. A function inside a function. Like an onion.

$$\boxed{\frac{d}{dm} f(g(m)) = f'(g(m)) \cdot g'(m)}$$

In words:

1. **Differentiate the outer layer, leaving the inside completely untouched.**
2. **Multiply by the derivative of the inside.**
3. Repeat if there are more layers.

3.5 2.5 The chain rule on the squared error

This is the derivative the whole book is built on. Take one observation, with x , y and c fixed and m the adjustable slope:

$$E(m) = (y - mx - c)^2$$

Outer layer is $(\cdot)^2$. By the power rule its derivative is $2(\cdot)$, inside untouched:

$$2(y - mx - c)$$

Inner layer is $y - mx - c$. As m changes, $-mx$ changes at rate $-x$; y and c are constants and contribute nothing:

$$-x$$

Multiply:

$$\boxed{\frac{dE}{dm} = -2x(y - mx - c)}$$

Chapter 0 §0E.5a reaches this same result by a completely different route — expanding the square first, then differentiating term by term:

$$E(m) = (y - c)^2 - 2(y - c)mx + m^2x^2 \implies \frac{dE}{dm} = -2(y - c)x + 2mx^2$$

Factor out $-2x$ and you get $-2x(y - mx - c)$. Two methods, one answer. That is what it looks like when the mathematics is not lying to you.

Put numbers in. With $x = 2$, $y = 10$, $c = 1$, $m = 1$:

$$\frac{dE}{dm} = -2(2)(10 - 2 - 1) = -4 \times 7 = -28$$

```
def error(m, c=1.0):
    return (10 - (m * 2 + c)) ** 2

def symbolic_gradient_m(x, y, m, c):
    return -2 * x * (y - m * x - c)

print(symbolic_gradient_m(2.0, 10.0, 1.0, 1.0))
print(numerical_slope(error, 1.0, h=1e-6))
```

-28.0

-27.999996007110894

And notice what -28 actually *tells* you: the error is falling steeply as m rises, so m is too small and should be increased. That is not an abstract number. That is an instruction.

3.6 2.6 Break it deliberately: the dropped inner derivative

The single most common chain rule mistake is forgetting step 2.

```
def broken_gradient_m(x, y, m, c):
    return 2 * (y - m * x - c)      # inner derivative (-x) omitted

print(broken_gradient_m(2.0, 10.0, 1.0, 1.0))
print(symbolic_gradient_m(2.0, 10.0, 1.0, 1.0))
```

14.0
-28.0

Wrong magnitude — and worse, **wrong sign**. A model using this would confidently turn the dial in exactly the wrong direction and watch its error climb.

Now look again at that output. **No error. No traceback. No red text.** Python can tell you that you did arithmetic; it cannot tell you that you did calculus incorrectly. This is the entire reason Part 8 exists.

3.7 Exercises, Part 2

2.1 [core] Differentiate by rule and verify each numerically: $7x^4$; $x^5 + 2x^2$; $10x + 3$; $\frac{1}{2}x^2 - 4x$; 6.

2.2 [core] By chain rule: $(3x + 1)^4$; $(5 - 2x)^3$; $(x^2 + 1)^2$. For the third, also expand the bracket first and differentiate term by term — confirm the two routes agree.

2.3 [proof] With $x = 4$, $y = 20$, $c = 2$, $m = 3$, compute $\frac{dE}{dm}$ by hand from the boxed formula, then confirm numerically. (Answer: -48 .)

2.4 [proof] Derive $\frac{dE}{dc}$ for $E = (y - mx - c)^2$ by chain rule. You should get $-2(y - mx - c)$. Explain in one sentence why there is no x in it.

4 Part 3 — The functions machine learning actually differentiates

Polynomials are the training ground. These are the functions you will meet in practice.

4.1 3.1 The exponential

$$\frac{d}{dx}e^x = e^x$$

The exponential is its own derivative. That is not a coincidence to be memorised — it is essentially the *definition* of the number e : the base for which the growth rate equals the current value.

Consequence, by chain rule:

$$\frac{d}{dx}e^{kx} = k e^{kx}, \quad \frac{d}{dx}e^{-x^2} = -2x e^{-x^2}$$

4.2 3.2 The logarithm

$$\frac{d}{dx} \ln x = \frac{1}{x} \quad (x > 0)$$

You will meet this every time a model works with log-transformed costs or a log-likelihood. Note the domain restriction — $\ln$ of zero or a negative number is undefined, and this is the origin of a great many `nan` values in real training runs.

By chain rule:

$$\frac{d}{dx} \ln(f(x)) = \frac{f'(x)}{f(x)}$$

4.3 3.3 The sigmoid, derived in full

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

It squashes any real number into $(0, 1)$. Its derivative is famously tidy, and here is why — write it as $\sigma(x) = (1 + e^{-x})^{-1}$ and apply the chain rule:

$$\begin{aligned} \sigma'(x) &= -1 \cdot (1 + e^{-x})^{-2} \cdot \frac{d}{dx}(1 + e^{-x}) \\ &= -1 \cdot (1 + e^{-x})^{-2} \cdot (-e^{-x}) \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} \end{aligned}$$

Now split that fraction deliberately:

$$\frac{e^{-x}}{(1 + e^{-x})^2} = \underbrace{\frac{1}{1 + e^{-x}}}_{\sigma(x)} \cdot \underbrace{\frac{e^{-x}}{1 + e^{-x}}}_{1 - \sigma(x)}$$

$$\sigma'(x) = \sigma(x) (1 - \sigma(x))$$

```
import numpy as np

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_prime(x):
    s = sigmoid(x)
    return s * (1 - s)

for x in [-6.0, -2.0, 0.0, 2.0, 6.0]:
    print(f"x={x:>5}  sigma'={sigmoid_prime(x):.6f}  "
          f"numeric={numerical_slope(sigmoid, x, 1e-6):.6f}")

x= -6.0  sigma'=0.002467  numeric=0.002467
x= -2.0  sigma'=0.104994  numeric=0.104994
x=  0.0  sigma'=0.250000  numeric=0.250000
x=  2.0  sigma'=0.104994  numeric=0.104994
x=  6.0  sigma'=0.002467  numeric=0.002467
```

4.4 3.4 Vanishing gradients, met early

Read that output column again. The derivative peaks at 0.25 in the middle and falls to 0.0025 at $x = \pm 6$ — a hundredfold collapse.

The derivative is the signal that tells a parameter which way to move. If it is 0.0025, the parameter barely moves; the model has effectively stopped learning in that region even though its predictions are wrong. This is the **vanishing gradient problem**, and it is the single reason ReLU displaced sigmoid in deep networks. You have now met it as a column of numbers rather than as jargon.

4.5 3.5 tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

Same S-shape as sigmoid but squashing into $(-1, 1)$, and its derivative peaks at 1 rather than 0.25 — which is why it trains somewhat better than sigmoid, and still not well enough.

4.6 3.6 ReLU, and a function that has no derivative at a point

$$\text{ReLU}(x) = \max(0, x)$$

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \\ \text{undefined} & x = 0 \end{cases}$$

At exactly $x = 0$ the function has a corner. Approach from the right and the slope is 1; from the left it is 0. The limit definition requires both to agree, and they do not. **The derivative does not exist there.**

Every framework nonetheless returns a number at zero — usually 0, sometimes 1. This is called a **subgradient**: a legal choice from the set of slopes that touch the corner without crossing the function. It is a convention, not a theorem, and it works in practice because landing exactly on 0.0 is vanishingly rare with real-valued data.

Say this out loud once so it stays: *not every function you use in ML is differentiable everywhere, and the library quietly makes a choice on your behalf.*

4.7 3.7 Softplus, and a small delight

$$\text{softplus}(x) = \ln(1 + e^x)$$

A smooth approximation to ReLU. Its derivative, by the log-then-chain rule:

$$\frac{d}{dx} \ln(1 + e^x) = \frac{e^x}{1 + e^x} = \frac{1}{1 + e^{-x}} = \sigma(x)$$

The derivative of softplus *is* the sigmoid. These functions are not a random grab-bag; they are relatives.

4.8 Exercises, Part 3

3.1 [core] Implement `sigmoid`, `sigmoid_prime`, and a finite-difference check. Sweep $x \in [-10, 10]$ and report the largest disagreement.

3.2 [core] Differentiate e^{-x^2} and $\ln(3x + 1)$ by chain rule; verify numerically.

3.3 [proof] Derive $\frac{d}{dx} \tanh x = 1 - \tanh^2 x$ from the quotient form. Verify at five points.

3.4 [stretch] Plot `numerical_slope(relu, x, 1e-6)` for x from -1 to 1 in steps of 0.001 . Describe what happens near zero and explain it in terms of the limit definition.

5 Part 4 — Curvature: the second derivative

5.1 4.1 The derivative of the derivative

If the derivative is the slope, the derivative of the derivative is the rate at which the slope is changing. Written $\frac{d^2y}{dx^2}$ or $f''(x)$.

Take the error function from Chapter 0, with $x = 2$, $y = 10$, $c = 1$ fixed:

$$E(m) = (10 - 2m - 1)^2 = (9 - 2m)^2 = 81 - 36m + 4m^2$$

$$E'(m) = -36 + 8m \qquad E''(m) = 8$$

Check: at $m = 1$, $E'(1) = -36 + 8 = -28$, matching Part 2. And the minimum is where the slope is zero: $-36 + 8m = 0$, so $m = 4.5$. Sanity-check that against the original — at $m = 4.5$, $9 - 2(4.5) = 0$, so the error is exactly zero. Correct.

5.2 4.2 Convexity: bowls and everything else

- $f'' > 0$ everywhere $\Rightarrow$ the curve bends upward $\Rightarrow$ a **bowl** $\Rightarrow$ **convex**.
- $f'' < 0$ everywhere $\Rightarrow$ bends downward $\Rightarrow$ a dome $\Rightarrow$ concave.
- $f'' = 0 \Rightarrow$ a straight line, no bend at all.

For our $E(m)$, $E'' = 8$: positive, constant, everywhere. A bowl.

5.3 4.3 Why this is the whole ballgame

Roll a ball into a bowl. It does not matter where you release it — it always settles at the same single lowest point.

Now take a wiggly function with three separate dips. Release three balls from three places and they settle in three *different* dips. Where you started determined where you finished.

Squared error for linear regression is always the bowl. That is why least squares has exactly one answer, why no one worries about “local minima” in linear regression, and why two people fitting the same model to the same data get the same coefficients regardless of where their optimiser started. When Part 5 says “positive semidefinite Hessian,” it means precisely this: a bowl in every direction at once.

5.4 4.4 Taylor expansion: everything looks like a parabola if you zoom in

$$f(x + \delta) \approx f(x) + f'(x)\delta + \frac{1}{2}f''(x)\delta^2$$

In words: to predict a function’s value a little way from where you are, take where you are, add the slope times the distance, then correct for the bend.

Two payoffs, both used later:

- **Newton’s method.** Fit the parabola, jump straight to *its* minimum, repeat. Faster than gradient descent per step, but needs f'' — which in many dimensions is an $n \times n$ matrix, and that is why gradient descent usually wins on cost.

- **Learning rates.** Section 7.4 shows the stable learning rate is set by the curvature. The second-order term above is where that comes from.

5.5 Exercises, Part 4

4.1 [core] Compute f'' for x^3 , e^x , $\ln x$, and $\sigma(x)$. Which are convex on all of their domain?

4.2 [core] Show $E(m) = 81 - 36m + 4m^2$ has its minimum at $m = 4.5$ by setting $E' = 0$. Confirm $E(4.5) = 0$.

4.3 [proof] Approximate e^x near 0 to first, second and third order. Plot the error of each against the true e^x over $[-1, 1]$.

6 Part 5 — Many dials at once

Everything so far assumed one adjustable number. Real regression has one per feature.

6.1 5.1 Partial derivatives

A **partial derivative** asks: *if I nudge just this one number and freeze everything else, how does the output change?* The symbol ∂ replaces d purely to signal “there are other variables here and I am holding them still.”

Take the two-parameter error surface:

$$E(m, c) = (10 - (2m + c))^2$$

By chain rule, with $x = 2$ and $y = 10$:

$$\frac{\partial E}{\partial m} = -2 \cdot 2 \cdot (10 - 2m - c) = -4(10 - 2m - c)$$

$$\frac{\partial E}{\partial c} = -2 \cdot (10 - 2m - c)$$

```
def error(m, c):
    prediction = m * 2 + c
    return (10 - prediction) ** 2

def partial_wrt_m(error_fn, m, c, h=1e-6):
    return (error_fn(m + h, c) - error_fn(m, c)) / h

def partial_wrt_c(error_fn, m, c, h=1e-6):
    return (error_fn(m, c + h) - error_fn(m, c)) / h

m, c = 1.0, 1.0
print("slope if we nudge m:", partial_wrt_m(error, m, c))
print("slope if we nudge c:", partial_wrt_c(error, m, c))

slope if we nudge m: -27.999996007110894
slope if we nudge c: -13.999999005420705
```

By hand: $-4(10 - 2 - 1) = -28$ and $-2(7) = -14$. They agree.

Note $\partial E / \partial m$ is twice $\partial E / \partial c$, and that is not an accident — m gets multiplied by $x = 2$ before it reaches the prediction, so nudging m has twice the effect of nudging c . **A feature’s scale controls how sensitive the error is to its weight.** This is the entire reason feature standardisation matters, arriving three chapters early.

6.2 5.2 The gradient

Stack the partials into a vector:

$$\nabla E = \begin{bmatrix} \partial E / \partial m \\ \partial E / \partial c \end{bmatrix} = \begin{bmatrix} -28 \\ -14 \end{bmatrix}$$

Read “ ∇ ” as “grad” or “del”. The gradient is not a new kind of object — it is a list of ordinary partial derivatives, one per dial.

Key fact: the gradient points in the direction of **steepest ascent** — the direction in which the function increases fastest. Therefore $-\nabla E$ points downhill, which is the direction you want to step when minimising error.

An intuition for why: the gradient collects how much each direction contributes to the increase. Moving so as to combine all those contributions positively is, by construction, the best you can do; any other direction wastes part of the step moving sideways along a contour.

6.3 5.3 The Jacobian

When the *output* is also a vector, one gradient is not enough — you need one per output. Stacking them gives the **Jacobian**: one row per output, one column per input.

$$J = \begin{bmatrix} \partial f_1 / \partial x_1 & \cdots & \partial f_1 / \partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_m / \partial x_1 & \cdots & \partial f_m / \partial x_n \end{bmatrix}$$

A gradient is just the Jacobian of a function with a single output, written as a column.

6.4 5.4 The Hessian

The matrix of *second* partial derivatives — curvature in every direction at once:

$$H = \begin{bmatrix} \partial^2 E / \partial m^2 & \partial^2 E / \partial m \partial c \\ \partial^2 E / \partial c \partial m & \partial^2 E / \partial c^2 \end{bmatrix}$$

For $E = (y - mx - c)^2$, differentiate the partials from 5.1 once more:

$$\frac{\partial^2 E}{\partial m^2} = 2x^2, \quad \frac{\partial^2 E}{\partial m \partial c} = 2x, \quad \frac{\partial^2 E}{\partial c^2} = 2$$

With $x = 2$:

$$H = \begin{bmatrix} 8 & 4 \\ 4 & 2 \end{bmatrix}$$

```
import numpy as np
H = np.array([[8.0, 4.0], [4.0, 2.0]])
print(np.linalg.eigvalsh(H))

[ 0. 10.]
```

6.5 5.5 Positive semidefinite, decoded

A symmetric matrix is **positive semidefinite (PSD)** when all its eigenvalues are ≥ 0 . That is the multi-dimensional version of “ $f'' \geq 0$ ”: the surface bends upward, or is flat, in every direction.

Our eigenvalues are 0 and 10. So the surface is a bowl in one direction and perfectly **flat** in another — a trough, not a bowl. And the flat direction is real: with a single data point you can trade slope against intercept endlessly and keep the prediction unchanged. One observation cannot pin down two parameters.

That zero eigenvalue is not a bug in the arithmetic. It is the geometry telling you the truth about your data. In Part 6 the same fact reappears as a singular matrix, and in Chapter 3 the cure appears as regularisation.

6.6 5.6 The multivariable chain rule, and the shape of backpropagation

When functions compose and the intermediate values are vectors, the chain rule becomes a product of Jacobians:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Backpropagation is exactly this, applied repeatedly from the output backwards to the input. There is no additional idea in it. When you eventually study neural networks, the calculus will already be finished — only the bookkeeping will be new.

6.7 Exercises, Part 5

5.1 [core] Add a third parameter to `error` and write `partial_wrt_third` by analogy, without looking at the other two.

5.2 [core] Compute ∇E at $(m, c) = (3, 2)$ by hand and numerically.

5.3 [proof] Derive the Hessian entries $2x^2$, $2x$, 2 by differentiating the partials. Verify numerically with nested finite differences.

5.4 [stretch] Add a second data point $(x, y) = (5, 20)$ so the total error is the sum of two squared errors. Recompute the Hessian. Are the eigenvalues still 0 and something? Explain what changed and why.

7 Part 6 — Matrix calculus and the normal equations

This is the part that makes Chapter 1 Day 5 feel like revision.

7.1 6.1 Differentiating with respect to a vector

If β is a vector of p weights, then $\nabla_{\beta} f$ means: take $\partial f / \partial \beta_j$ for every j , and stack them into a vector of length p . It is bookkeeping, not a new operation.

7.2 6.2 Four identities you will use forever

Each is verifiable numerically, and you should verify each rather than trusting the page.

Expression	Gradient with respect to β	Scalar analogue
$a^{\top} \beta$	a	$\frac{d}{dx}(ax) = a$
$\beta^{\top} A \beta$	$(A + A^{\top})\beta$, $= 2A\beta$ if A symmetric	$\frac{d}{dx}(ax^2) = 2ax$
$\ \beta\ ^2 = \beta^{\top} \beta$	2β	$\frac{d}{dx}(x^2) = 2x$
$\ y - X\beta\ ^2$	$-2X^{\top}(y - X\beta)$	$\frac{d}{dm}(y - mx)^2 = -2x(y - mx)$

Read the right-hand column. **Every matrix identity is the scalar rule you already know, with transposes inserted to make the shapes line up.** That is the whole of matrix calculus at this level. Nothing new is being asked of you.

7.3 6.3 Deriving the normal equations, start to finish

The least-squares objective:

$$E(\beta) = \|y - X\beta\|^2 = (y - X\beta)^{\top}(y - X\beta)$$

Step 1 — expand.

$$E(\beta) = y^{\top}y - 2y^{\top}X\beta + \beta^{\top}X^{\top}X\beta$$

(The two middle terms are equal because each is a scalar and a scalar equals its own transpose.)

Step 2 — differentiate term by term, using the table above:

- $y^{\top}y$ has no β in it, so its gradient is $\mathbf{0}$.
- $-2y^{\top}X\beta$ is of the form $a^{\top}\beta$ with $a^{\top} = -2y^{\top}X$, so its gradient is $-2X^{\top}y$.
- $\beta^{\top}(X^{\top}X)\beta$ is a quadratic form with a symmetric matrix, so its gradient is $2X^{\top}X\beta$.

$$\nabla_{\beta} E = -2X^{\top}y + 2X^{\top}X\beta$$

Step 3 — set the gradient to zero. The minimum of a bowl is where the slope vanishes:

$$-2X^{\top}y + 2X^{\top}X\beta = 0 \implies \boxed{X^{\top}X\beta = X^{\top}y}$$

These are the **normal equations**. Solving them:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Step 4 — verify the hand derivation numerically. Never skip this.

```
import numpy as np

X = np.array([[12.0, 15.0],
              [30.0, 25.0],
              [ 5.0,  8.0],
              [40.0, 45.0],
              [15.0, 12.0]])
y = np.array([12.0, 30.0, 8.0, 45.0, 15.0])

def E(beta):
    r = y - X @ beta
    return r @ r

def grad_analytic(beta):
    return -2 * X.T @ (y - X @ beta)

def grad_numeric(beta, h=1e-6):
    g = np.zeros_like(beta)
    for j in range(len(beta)):
        step = np.zeros_like(beta); step[j] = h
        g[j] = (E(beta + step) - E(beta - step)) / (2 * h)
    return g

beta = np.array([0.5, 0.5])
print("analytic:", grad_analytic(beta))
print("numeric :", grad_numeric(beta))

beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
print("beta_hat:", beta_hat)
print("gradient at the solution:", grad_analytic(beta_hat))

analytic: [-374. -365.]
numeric  : [-374. -365.]
beta_hat: [0.67362609 0.39305969]
gradient at the solution: [-4.15667500e-13 -2.27373675e-13]
```

The hand-derived gradient matches the measured one. And at $\hat{\beta}$ the gradient is zero to within floating-point dust — which is exactly what “we found the bottom of the bowl” looks like numerically.

This is Chapter 1, Day 5, done early. When you get there, you will be revising.

7.4 6.4 Break it deliberately: collinearity

```
X_bad = np.column_stack([X[:, 0], 2 * X[:, 0]]) # second column is twice the first
print(np.linalg.eigvalsh(X_bad.T @ X_bad))
np.linalg.solve(X_bad.T @ X_bad, X_bad.T @ y)

[ 0. 14470.]
LinAlgError: Singular matrix
```

One eigenvalue is exactly zero, so $X^\top X$ cannot be inverted, and the solve fails outright.

This is the same flat direction from section 5.5, now wearing different clothes. If one feature is a copy of another, infinitely many weight vectors produce identical predictions, and asking for “the” answer is asking a question with no unique answer. The matrix is not being difficult. It is being honest.

7.5 6.5 Ridge, as a one-line consequence

Add a penalty on the size of β :

$$E_{\text{ridge}}(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2$$

Differentiate — sum rule, plus the third identity from 6.2:

$$\nabla_{\beta} E_{\text{ridge}} = -2X^\top(y - X\beta) + 2\lambda\beta$$

Set to zero:

$$(X^\top X + \lambda I) \beta = X^\top y$$

Adding λ to the diagonal lifts every eigenvalue by λ . The zero eigenvalue from the breakage above becomes λ , the matrix becomes invertible, and the problem has a unique answer again. Ridge regression is not a trick bolted on from outside — it falls out of one extra term and the sum rule.

7.6 Exercises, Part 6

6.1 [core] Verify all four identities in 6.2 numerically for random β of length 3.

6.2 [proof] Reproduce the Step 1–3 derivation on paper without looking. Then check your gradient numerically on the data above.

6.3 [core] Solve the ridge equations for $\lambda = 0.1, 1, 10$ on `X_bad`. Show the solve succeeds and describe what happens to $\hat{\beta}$ as λ grows.

6.4 [stretch] Derive ∇_{β} for the lasso penalty $\lambda\|\beta\|_1$ and explain why it causes trouble at $\beta_j = 0$. (Hint: section 3.6.)

8 Part 7 — Optimisation, which is what calculus is *for*

8.1 7.1 Critical points

Setting the gradient to zero finds *candidates* for the optimum, not the optimum. A zero gradient means one of:

- a **minimum** — Hessian PSD (bowl),
- a **maximum** — Hessian negative semidefinite (dome),
- a **saddle** — Hessian has both positive and negative eigenvalues (bowl one way, dome the other).

For convex problems only the first is possible, which is why linear regression can be solved by setting the gradient to zero and never worrying about which kind of point you landed on.

8.2 7.2 Gradient descent

When you cannot solve $\nabla E = 0$ algebraically — which is nearly always, outside linear regression — you walk downhill instead:

1. Start somewhere.
2. Compute the gradient.
3. Step a little way in the direction $-\nabla E$.
4. Repeat until the gradient is near zero.

```
m, c = 0.0, 0.0
learning_rate = 0.01

for step in range(50):
    grad_m = partial_wrt_m(error, m, c)
    grad_c = partial_wrt_c(error, m, c)
    m -= learning_rate * grad_m
    c -= learning_rate * grad_c

print(f"m={m:.3f}, c={c:.3f}, error={error(m, c):.6f}")

m=3.979, c=1.990, error=0.002656
```

The prediction $m \cdot 2 + c$ is now 9.948, closing on the target of 10 — fifty steps is not quite enough at this learning rate, which is itself worth noticing. Note it did not find “the” answer — with one data point there is a whole line of (m, c) pairs that work, exactly the flat direction from 5.5. It found *an* answer, and which one depends on where it started.

8.3 7.3 Break it deliberately: the learning rate

```
m, c = 0.0, 0.0
learning_rate = 5.0          # was 0.01

for step in range(6):
    grad_m = partial_wrt_m(error, m, c)
    grad_c = partial_wrt_c(error, m, c)
    m -= learning_rate * grad_m
```

```

c -= learning_rate * grad_c
print(f"{step}  m={m:>14.2f}  c={c:>14.2f}  error={error(m, c):.4e}")
0  m=      200.00  c=      100.00  error=2.4010e+05
1  m=     -9600.00  c=     -4800.00  error=5.7648e+08
2  m=    470600.05  c=    235300.03  error=1.3841e+12
3  m= -23059673.38  c= -11529836.69  error=3.3234e+15
4  m= 1126940326.62  c= 568470163.31  error=7.9657e+18
5  m=-55193059673.38  c=-25031529836.69  error=1.8338e+22

```

The error does not shrink. It **explodes** — twenty-five orders of magnitude in six steps.

The geometry: the step overshoots the bottom of the valley and lands further up the *other* side than it started. The next gradient is therefore larger, so the next overshoot is worse, and the process feeds itself.

The diagnostic habit to build now:

- Error rising, especially fast $\Rightarrow$ learning rate too large.
- Error falling but glacially $\Rightarrow$ learning rate too small.
- Error becomes **nan** $\Rightarrow$ it already exploded past the largest representable float.

8.4 7.4 How large is too large, exactly

This is where Part 4 earns its keep. For a quadratic with largest Hessian eigenvalue L , gradient descent converges only if

$$\eta < \frac{2}{L}$$

For the single-variable version $E(m) = 81 - 36m + 4m^2$, we have $E'' = L = 8$, so the threshold is $\eta < 0.25$. Test it:

η	after 60 steps
0.01	$m = 4.47$ (converging)
0.20	$m = 4.50$ (converged)
0.25	$m = 0$ (oscillates forever, never settles)
0.30	$m \approx -2.6 \times 10^9$ (diverged)
5.00	$m \approx -1.3 \times 10^{96}$ (violently diverged)

At exactly $\eta = 2/L$ the iteration bounces between two points forever without converging or diverging — the knife edge, visible in the table.

So “the learning rate is a hyperparameter you tune by trial and error” is only half true. It is bounded above by the curvature of your loss surface, and that is a fact you can compute.

8.5 7.5 Variants, named only

- **Batch** gradient descent uses all data for each step (what we did above).
- **Stochastic (SGD)** uses one sample per step — noisier, far cheaper, and the noise sometimes helps escape bad regions in non-convex problems.

- **Mini-batch** uses a small chunk; the practical default everywhere.
- **Momentum** adds a fraction of the previous step to the current one, so the path builds speed along consistent directions and damps oscillation across a narrow valley.

You need these as vocabulary now, not as mastery.

8.6 7.6 Constrained optimisation, briefly

Sometimes you minimise subject to a constraint — “make the error small, but keep $\|\beta\|^2$ below some budget.” At the solution, the gradient of the objective and the gradient of the constraint are parallel, and the constant of proportionality is called a **Lagrange multiplier**, written λ .

The payoff: **ridge and lasso are constrained least squares**. The λ in section 6.5 is exactly that multiplier. Chapter 3’s penalties are not arbitrary inventions; they are the Lagrangian form of “keep the coefficients small.”

8.7 Exercises, Part 7

7.1 [core] Run the descent loop with $\eta = 0.001, 0.01, 0.1, 0.2$ and plot error against step for each on one chart.

7.2 [proof] For $E(m) = 81 - 36m + 4m^2$, find the largest stable η by bisection, and confirm it matches $2/E''$.

7.3 [core] Modify the descent loop to stop when $\|\nabla E\| < 10^{-6}$ instead of after a fixed count. Report how many steps each learning rate needs.

7.4 [stretch] Implement momentum and compare step counts against plain descent on the same problem.

9 Part 8 — Doing calculus without doing calculus

9.1 8.1 Forward versus central differences

Everything so far used the **forward** difference:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

The **central** difference is better for the same h :

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Forward difference error shrinks like h ; central like h^2 . Same cost, one extra evaluation, substantially better accuracy. Use central differences for checking; use forward only when $f(x)$ is already computed and you want the extra evaluation for free.

9.2 8.2 The gradient check — build this and keep it

```
import numpy as np

def gradient_check(f, grad_f, x, h=1e-6):
    """Compare an analytic gradient against central differences.
    Returns the relative error. Below 1e-7 is fine; above 1e-4 is a bug."""
    x = np.asarray(x, dtype=float)
    analytic = np.asarray(grad_f(x), dtype=float)
    numeric = np.zeros_like(x)
    for j in range(x.size):
        step = np.zeros_like(x); step[j] = h
        numeric[j] = (f(x + step) - f(x - step)) / (2 * h)
    denom = np.linalg.norm(analytic) + np.linalg.norm(numeric)
    return np.linalg.norm(analytic - numeric) / max(denom, 1e-30)
```

Applied to the Part 6 gradient, and to the deliberately broken one:

```
def grad_broken(beta):
    return -1 * X.T @ (y - X @ beta)    # dropped the factor of 2

print("correct:", gradient_check(E, grad_analytic, np.array([0.5, 0.5])))
print("broken :", gradient_check(E, grad_broken, np.array([0.5, 0.5])))

correct: 5.773983625472507e-12
broken : 0.333333333333289755
```

A relative error of 10^{-12} says the derivation is right. A relative error of 0.33 says it is wrong, loudly, in a way no traceback would ever have told you. **Run this every single time you derive a gradient by hand.** It is the reason Part 2's silent failure cannot survive contact with your workflow.

9.3 8.3 Symbolic differentiation with SymPy

```
import sympy as sp

m, c, x, y = sp.symbols('m c x y')
E = (y - m*x - c)**2
print(sp.simplify(sp.diff(E, m)))

2*x*(c + m*x - y)
```

Which is $-2x(y - mx - c)$ with the sign folded inside. Same answer as section 2.5.

Use SymPy as a **checking tool, not a replacement for understanding**. It will happily hand you a correct derivative you cannot interpret, and an uninterpretable gradient is not much use when the question is *why is my model not learning*.

9.4 8.4 A tiny autodiff engine, from scratch

This is the “no black boxes” payoff. Frameworks compute gradients by building a graph of operations and walking it backwards, applying the chain rule at each node. Here is the whole idea in forty lines.

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():
            self.grad += out.grad          # sum rule
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')
        def _backward():
            self.grad += other.data * out.grad    # product rule
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def __pow__(self, n):
        out = Value(self.data ** n, (self,), f'**{n}')
        def _backward():
            self.grad += n * (self.data ** (n - 1)) * out.grad    # power + chain
```

```

    out._backward = _backward
    return out

def __neg__(self):          return self * -1
def __sub__(self, other):   return self + (-other)
def __rsub__(self, other):  return (-self) + other
def __radd__(self, other):  return self + other
def __rmul__(self, other):  return self * other

def backward(self):
    topo, visited = [], set()
    def build(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build(child)
            topo.append(v)
    build(self)
    self.grad = 1.0
    for v in reversed(topo):
        v._backward()

```

Use it on the error from section 2.5:

```

m = Value(1.0)
c = Value(1.0)
E = (Value(10.0) - (m * 2.0 + c)) ** 2
E.backward()
print("dE/dm =", m.grad)
print("dE/dc =", c.grad)

dE/dm = -28.0
dE/dc = -14.0

```

The same -28 and -14 you derived by hand in Part 5, now computed by a machine that knows only three rules: sum, product, power — each of which you derived yourself in Part 2.

`loss.backward()` in any framework is this, scaled up. It is not magic. You just wrote it.

9.5 8.5 Integration, the minimum viable dose

Differentiation and integration are inverses: the derivative measures rate, the integral accumulates it. Where regression needs integrals:

- A **probability density** integrates to 1 over its whole range.
- An **expectation** is an integral: $\mathbb{E}[X] = \int x p(x) dx$.
- A **cumulative distribution** is the integral of a density up to a point.
- **Survival analysis** (Chapter 6) is built on the cumulative hazard, which is an integral of the hazard rate.

You will rarely compute one by hand in this book. You do need to recognise what the symbol is claiming.

```
import numpy as np
xs = np.linspace(-1, 1, 100001)
density = np.exp(-xs**2 / 2) / np.sqrt(2 * np.pi)
print(np.trapezoid(density, xs))

0.6826894921209545
```

That is the familiar “68% of a normal distribution lies within one standard deviation” — computed, not quoted.

9.6 Exercises, Part 8

8.1 [core] Compare forward and central differences on x^3 at $x = 2$ across h from 10^{-1} to 10^{-10} . Plot both error curves on log axes.

8.2 [core] Add `exp` and `log` nodes to the `Value` class. Verify each against finite differences.

8.3 [proof] Use `gradient_check` on the ridge gradient from 6.5. Then break it by dropping the $2\lambda\beta$ term and confirm the check catches it.

8.4 [stretch] Use the `Value` class to run gradient descent on the two-parameter problem, and confirm it reaches the same answer as section 7.2.

10 Appendix A — Formula sheet

Definitions

$$\frac{dy}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad \nabla E = \left[\frac{\partial E}{\partial \theta_1}, \dots, \frac{\partial E}{\partial \theta_p} \right]^\top$$

Rules

Rule	Statement
Power	$\frac{d}{dx} x^n = nx^{n-1}$
Constant multiple	$\frac{d}{dx} [af] = af'$
Sum	$\frac{d}{dx} [f+g] = f' + g'$
Product	$\frac{d}{dx} [fg] = f'g + fg'$
Quotient	$\frac{d}{dx} \left[\frac{f}{g} \right] = \frac{f'g - fg'}{g^2}$
Chain	$\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$

Functions

$f(x)$	$f'(x)$
e^x	e^x
$\ln x$	$1/x$
$\sigma(x)$	$\sigma(x)(1 - \sigma(x))$
$\tanh x$	$1 - \tanh^2 x$
$\text{ReLU}(x)$	1 if $x > 0$, 0 if $x < 0$, undefined at 0
$\ln(1 + e^x)$	$\sigma(x)$

Matrix gradients

Expression	∇_β
$a^\top \beta$	a
$\beta^\top A \beta$ (A sym.)	$2A\beta$
$\ \beta\ ^2$	2β
$\ y - X\beta\ ^2$	$-2X^\top(y - X\beta)$

Results

$$X^\top X \beta = X^\top y \quad (X^\top X + \lambda I) \beta = X^\top y \quad \eta < \frac{2}{\lambda_{\max}(H)}$$

11 Appendix B — Notation

Symbol	Name	Meaning
h	step size	the shrinking gap in a numerical derivative
$\frac{dy}{dx}, f'(x)$	derivative	slope at a point
$f''(x)$	second derivative	rate of change of the slope; curvature
∂	partial	derivative holding other variables fixed
∇	gradient	vector of all partials
J	Jacobian	matrix of partials, one row per output
H	Hessian	matrix of second partials
η	learning rate	step size in gradient descent
λ	multiplier / penalty	Lagrange multiplier; regularisation strength
$\int$	integral	accumulated area
PSD	positive semidefinite	all eigenvalues ≥ 0 ; a bowl in every direction

12 Appendix C — Error decoder

Symptom	Likely cause	Check
Numerical slope is 0.0	h below floating-point resolution	raise h to 10^{-6}
Numerical slope is noisy	h too small	central differences, $h \approx 10^{-6}$
Loss grows every step	learning rate above $2/L$	compute $\lambda_{\max}(H)$, or halve η
Loss becomes nan	already diverged past float range	lower η ; check for $\ln(0)$
Loss barely moves	η too small, or saturated sigmoid	check gradient magnitudes
Gradient check $> 10^{-4}$	derivation is wrong	look for a dropped chain-rule factor
LinAlgError: Singular matrix	collinear features	inspect eigenvalues; add ridge λ
nan from a correlation	zero-variance feature	check <code>.std()</code> before dividing
No error, but wrong answer	calculus mistake	this is what gradient checking is for

13 Appendix D — Exit checks

Part 1–2. Explain what a derivative is in one sentence. Compute one numerically and justify $h \approx 10^{-6}$. Apply the power, constant, sum, and chain rules. Derive $\frac{dE}{dm} = -2x(y - mx - c)$ from memory.

Part 3–4. Derive $\sigma' = \sigma(1 - \sigma)$. Explain vanishing gradients using the numbers, not the phrase. Say what $f'' > 0$ means and why regression cares.

Part 5–6. Explain a partial derivative to someone who has never heard the term. Say what PSD means geometrically. Derive the normal equations start to finish without notes.

Part 7–8. Explain what gradient descent does and why too large a learning rate breaks it, with the $2/L$ bound. Write `gradient_check` from scratch. Explain what `loss.backward()` is doing.

If any of these is a “no”, return to that part before going on. Chapter 1 will not re-teach them; it will simply use them.